Stacklyst

Documento de Visão

Laboratório de Engenharia de Software
Projeto acadêmico — Laboratório de Engenharia de Software
Data da revisão: 12/09/2026

1. Índice

1. Índice 2
2. Objetivo 3
3. Necessidade do Negócio 3
4. Descrição do Escopo 3
4.1 Dentro do escopo 3
4.2 Fora do escopo da primeira versão 5
5. Equipe 5
6. Especificações Técnicas 6
7. Riscos 7
8. Cronograma de Marcos Resumido 8
9. Orçamento Resumido 9

2. Objetivo

O Stacklyst é uma plataforma acadêmica gamificada para ensino, prática e evolução em programação. O produto reúne trilhas, lições, exercícios, desafios, duelos, publicações, mensagens e acompanhamento de progresso em um ambiente integrado destinado principalmente a estudantes e desenvolvedores em formação ou aperfeiçoamento.

O problema central é a fragmentação entre aprender conceitos, praticar código, manter constância e demonstrar evolução. O objetivo educacional é favorecer prática ativa, feedback frequente e progressão observável; o objetivo da plataforma é conectar essas experiências a interações comunitárias e, quando autorizado, a oportunidades profissionais.

A gamificação utiliza XP, níveis, sequência de atividades, conquistas, rankings e faixas como mecanismos de engajamento. A ajuda durante os exercícios usa dicas, documentação e autocomplete definidos no próprio conteúdo. As decisões competitivas que exigem revisão são encaminhadas a um avaliador humano autorizado.

3. Necessidade do Negócio

Para fins acadêmicos, a equipe do Stacklyst exerce o papel de cliente e proprietária do produto. O negócio oferece uma plataforma educacional voltada à aprendizagem prática de programação para estudantes e desenvolvedores em formação.

O cenário observado contém plataformas muito passivas, exercícios pouco variados, dificuldade de manter frequência nos estudos, pouca interação entre estudantes, distância entre conteúdo e situações práticas, dificuldade para medir a evolução e pouca conexão entre aprendizagem, interação entre pessoas e oportunidades profissionais.

O Stacklyst pretende melhorar esse cenário com percursos organizados, atividades interativas, execução de código, feedback, desafios competitivos, avaliação humana quando necessária, progressão gamificada, espaços comunitários e um módulo de oportunidades. A solução mantém uma separação entre evidência educacional e decisão de recrutamento: desempenho público pode ser um indicador, mas não uma decisão automática.

4. Descrição do Escopo

4.1 Dentro do escopo

Módulo	Descrição	Estado	Evidência
Autenticação e usuários	Cadastro, login, logout, sessão, OAuth e controle de papéis.	Implementado	src/app/api/auth; src/lib/auth.ts; src/lib/supabase
Perfil	Dados públicos do desenvolvedor, histórico de publicações e indicadores de evolução.	Implementado	src/app/profile/[username]; src/app/api/profile

Módulo	Descrição	Estado	Evidência
Trilhas	Catálogo versionado com unidades, lições e exercícios; mapa e lista compartilham os nós e estados.	Implementado	src/lib/learning/catalog.ts; src/app/trails/LearningJourney.tsx
Lições	Páginas próprias com conteúdo e etapas interativas.	Implementado	src/app/lesson/[lessonId]; src/lib/lessons
Exercícios	Avaliação no servidor com respostas canônicas, ajuda do exercício e execução remota de código.	Implementado	src/lib/learning/assessment.ts; src/lib/exercises/evaluator.ts
Desafios e execução de código	Problemas de programação, envio de solução e execução em serviço externo.	Implementado	src/app/api/run; src/lib/code-runner.ts
Duelos	Criação de duelo aberto, entrada em vaga na arena, envio de solução e resolução competitiva.	Implementado parcialmente	src/app/duels; src/services/duel.service.ts
Matchmaking	Entrada em duelos PENDING na arena ou convite por faixa de XP.	Implementado	src/services/duel.service.ts; src/app/api/duels/route.ts; src/lib/duels/lifecycle.ts
Avaliações	Avaliação automática e fluxo de avaliação humana para duelos.	Implementado parcialmente	src/app/api/evaluations; src/services/evaluator.service.ts
Avaliadores	Elegibilidade, candidatura, aprovação administrativa e parecer técnico.	Implementado	src/app/evaluators; src/app/evaluations; src/services/evaluator.service.ts
XP, níveis e sequência	Pontuação transacional, níveis e ofensiva por atividade.	Implementado	src/services/xp.service.ts; src/lib/streak.ts
Conquistas	Badges por critérios de atividade e sequência.	Implementado	prisma/schema.prisma; src/services/xp.service.ts
Ranking e divisões	Ranking por XP e faixas Bronze a Diamante; não implementa rating Elo.	Implementado	src/lib/learning/rewards.ts; language-xp.ts
Publicações e interação	Feed, publicações, respostas, reações, votos, favoritos, seguidores e mensagens.	Implementado	src/app/feed; src/services/post.service.ts; src/app/api/posts
Eventos	Listagem, criação e participação em eventos/desafios comunitários.	Implementado parcialmente	src/app/events; src/services/event.service.ts

Módulo	Descrição	Estado	Evidência
Notificações	Central web e tipos de eventos da plataforma.	Implementado na web	src/app/notifications; src/services/notification.service.ts
Vagas e empresas	Cadastro de empresa por usuário autenticado, publicação de vagas, consulta e atualização de etapas de candidatura.	Implementado parcialmente	src/app/jobs; src/app/recruiter; src/services/job.service.ts
Administração	Usuários, papéis, denúncias, candidaturas de avaliador, métricas e quiz diário baseado em conteúdo curado.	Implementado parcialmente	src/app/admin; src/app/api/admin
Acompanhamento de progresso	Tentativas, XP, níveis, ofensiva, checkpoints e atividade semanal.	Implementado	QuizAttempt, LanguageTrail; src/app/api/quiz/weekly-activity

4.2 Fora do escopo da primeira versão

- Marketplace de cursos, serviços ou produtos digitais.
- Contratação, assinatura de contrato e pagamento dentro do Stacklyst.
- IDE online completa equivalente a um ambiente desktop, com depurador e gerenciamento avançado de projetos.
- Sistema próprio de videoconferência.
- Emissão de certificações oficiais reconhecidas por órgãos externos.
- Integração imediata com dezenas de plataformas externas.
- Aplicativo móvel nativo; a versão atual é uma aplicação web responsiva.

Esses itens não fazem parte da versão documentada em 12/09/2026.

5. Equipe

Campo	Preenchimento
Nome	Guilherme Alves de Campos
Formação	Graduando em Análise e Desenvolvimento de Sistemas
Experiência	Desenvolvimento backend e lógica de negócio no projeto Stacklyst.
Papel no projeto	Desenvolvimento backend e lógica de negócio
Responsabilidades	APIs, serviços, persistência de dados, regras de negócio e testes do backend.

Campo	Preenchimento
Nome	Paulo Henrique Leal dos Santos

Campo	Preenchimento
Formação	Graduando em Análise e Desenvolvimento de Sistemas
Experiência	Desenvolvimento backend e lógica de negócio no projeto Stacklyst.
Papel no projeto	Desenvolvimento backend e lógica de negócio
Responsabilidades	APIs, serviços, persistência de dados, regras de negócio e testes do backend.

Campo	Preenchimento
Nome	Pedro Antonio Silvestre Tescaro
Formação	Graduando em Análise e Desenvolvimento de Sistemas
Experiência	Design de interface e desenvolvimento frontend no projeto Stacklyst.
Papel no projeto	Design de interface e desenvolvimento frontend
Responsabilidades	Experiência do usuário, protótipos, componentes visuais, responsividade e integração das telas com as APIs.

6. Especificações Técnicas

As tecnologias abaixo foram verificadas no repositório e na implantação de produção do Stacklyst.

Camada	Especificação verificada	Evidência
Frontend Web	Next.js 16.3.2 (App Router), React 19.2.4, TypeScript 5, Tailwind CSS 4, CodeMirror e Tiptap.	package.json; src/app
Backend	Route Handlers do Next.js em runtime Node, serviços TypeScript, validação Zod e controles de taxa.	src/app/api; src/services; src/lib/api-handler.ts
Banco de Dados	PostgreSQL hospedável no Supabase, modelado pelo Prisma ORM 7.9.1.	prisma/schema.prisma; docs/DATABASE.md
Autenticação	Supabase Auth/SSR com sessão verificada e camada server-only de JWT da aplicação.	src/lib/supabase; src/lib/auth.ts; src/lib/jwt.ts
Infraestrutura	Docker/Docker Compose; documentação de Vercel e Supabase; Upstash Redis opcional para rate limit distribuído.	Dockerfile; docker-compose*.yml; docs/DEPLOYMENT.md
APIs	REST/JSON internas; Supabase; execução de código por Wandbox/Judge0.	src/app/api; src/lib/code-runner.ts
Controle de versão	Git, GitHub, Conventional Commits, Husky, lint-staged e CI configurável.	.git; commitlint.config.js; .husky; package.json

Camada	Especificação verificada	Evidência
Arquitetura geral	Aplicação web modular: UI e Server Components → API Routes → serviços/domínio → Prisma/PostgreSQL; integrações externas isoladas em bibliotecas.	docs/ARCHITECTURE.md; estrutura src

A aplicação entregue é web responsiva. Um cliente móvel nativo exigiria um novo projeto e não integra o escopo atual.

7. Riscos

Risco	Categoria	Prob.	Impacto	Nível	Estratégia	Contingência
Atraso de desenvolvimento	Gerencial	Alta	Alto	Alto	Planejar entregas incrementais e priorizar o MVP.	Reduzir itens de prioridade média/baixa e replanejar marcos.
Mudança frequente de requisitos	Gerencial	Alta	Alto	Alto	Manter baseline versionado e registrar decisões.	Executar análise de impacto e nova validação acadêmica.
Escopo excessivo	Gerencial	Alta	Alto	Alto	Separar MVP, expansão e itens fora do escopo.	Congelar novas funcionalidades até concluir o núcleo.
Dependência de serviços externos	Técnico	Média	Alto	Alto	Definir timeout e tratamento de falhas para autenticação, banco e executor de código.	Manter mensagens de erro e impedir confirmação de progresso quando a operação externa falhar.
Indisponibilidade do executor de código	Técnico	Média	Alto	Alto	Aplicar limites, testes definidos no servidor e tratamento de timeout.	Não aprovar a resposta nem conceder XP; manter o código disponível para nova tentativa.
Custo de infraestrutura	Organizacional	Média	Alto	Alto	Monitorar uso e definir cotas por serviço.	Reduzir recursos opcionais e migrar para plano compatível.
Falha de segurança	Técnico	Média	Alto	Alto	RBAC, validação, rate limit, segredo server-only e testes.	Revogar credenciais, conter acesso e corrigir vulnerabilidade.
Exposição indevida de dados	Técnico	Média	Alto	Alto	Definir campos públicos e consentimento para recrutamento.	Restringir consultas, registrar incidente e revisar permissões.

Risco	Categoria	Prob.	Impacto	Nível	Estratégia	Contingência
Baixa adesão	Externo	Média	Alto	Alto	Testar usabilidade e valor das trilhas com usuários.	Simplificar onboarding e ajustar mecânicas com feedback.
Complexidade do matchmaking	Técnico	Alta	Médio	Alto	Começar com regras determinísticas e métricas.	Usar desafio direto e busca ampliada como alternativa.
Falha na avaliação automática de código	Técnico	Média	Alto	Alto	Sandbox externo, timeout, casos de teste e fallback humano.	Suspender pontuação e encaminhar para revisão.
Problemas de desempenho	Técnico	Média	Alto	Alto	Paginação, índices, cache e testes de carga.	Reduzir consultas, limitar listas e escalar serviços críticos.
Indisponibilidade dos integrantes	Organizacional	Média	Alto	Alto	Distribuir conhecimento e registrar decisões.	Repriorizar entregas e redistribuir responsabilidades.
Incompatibilidade entre navegadores	Técnico	Média	Médio	Médio	Testar Chrome, Edge, Firefox e Safari nas versões suportadas.	Corrigir o fluxo afetado e documentar limitação temporária.
Moderação insuficiente	Gerencial	Média	Alto	Alto	Denúncia, fila, critérios e auditoria administrativa.	Ocultar conteúdo, restringir usuário e ampliar revisão.
Fraude em ranking ou duelo	Técnico	Alta	Alto	Alto	Idempotência, validação server-side e registros de auditoria.	Anular resultado, recalcular pontuação e bloquear abuso.

8. Cronograma de Marcos Resumido

O cronograma abaixo foi consolidado a partir do histórico Git e da revisão documental. Períodos amplos representam fases iterativas do projeto.

Marco	Data
Início do projeto	16/06/2026
Levantamento de requisitos	Junho a agosto de 2026
Documento de visão	21/08 a 12/09/2026
Atividades do negócio	21/08 a 12/09/2026
Requisitos do sistema	21/08 a 12/09/2026

Marco	Data
Casos de uso	21/08 a 12/09/2026
Prototipação	Junho a setembro de 2026
Modelagem do banco	Junho a agosto de 2026
Definição de arquitetura	Junho a agosto de 2026
MVP	Junho a agosto de 2026
Módulo de autenticação	Junho a setembro de 2026
Trilhas e lições	Junho a setembro de 2026
Gamificação	Junho a setembro de 2026
Duelos	Junho a setembro de 2026
Publicações e interação	Junho a setembro de 2026
Vagas e empresas	Agosto a setembro de 2026
Testes	Junho a setembro de 2026
Testes de aceitação	Setembro de 2026
Correções	Contínuo até a entrega
Deploy	Produção ativa em setembro de 2026
Entrega final	12/09/2026

9. Orçamento Resumido

Estimativa acadêmica de custo direto: R$ 0,00, considerando equipamentos próprios, trabalho da equipe e planos gratuitos atualmente utilizados.

Categoria	Item	Valor	Observação
Custos fixos	Hardware	R$ 0,00	Equipamentos próprios da equipe.
Custos fixos	Licenças de software	R$ 0,00	Ferramentas e bibliotecas gratuitas ou de código aberto.
Custos fixos	Treinamentos	R$ 0,00	Aprendizado realizado no contexto acadêmico.
Infraestrutura	Hospedagem e domínio	R$ 0,00	Implantação atual no endereço gratuito stacklyst.vercel.app.

Categoria	Item	Valor	Observação
Infraestrutura	Banco de dados e armazenamento	R$ 0,00	Uso acadêmico no plano gratuito do Supabase.
Infraestrutura	APIs e execução de código	R$ 0,00	Uso atual dentro das cotas gratuitas disponíveis.
Infraestrutura	Recursos de assistência editorial	Não aplicável	Os recursos de assistência editorial são executados pela própria aplicação, sem custo externo adicional.
Custos variáveis	Desenvolvimento	R$ 0,00	Projeto acadêmico sem remuneração contabilizada.
Custos variáveis	Pessoal de apoio	R$ 0,00	Sem contratação externa.
Custos variáveis	Encargos e benefícios	Não aplicável	Não há vínculo empregatício no projeto acadêmico.
Custos variáveis	Instalações, energia, materiais e rede	R$ 0,00 adicional	Recursos pessoais já disponíveis; consumo não segregado.
Qualidade	Testes e homologação	R$ 0,00	Ferramentas automatizadas e validação acadêmica.
Riscos	Margem de contingência	R$ 0,00	Sem desembolso previsto na versão acadêmica.
Financeiro	Margem de lucro	Não aplicável	Projeto acadêmico sem comercialização nesta versão.